LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack And Queue

Oleh:

Muhammad Hafi NIM. 2510817310011

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
April 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: stack and queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Muhammad Hafi
NIM : 25108173100111

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
Soal 1: Implementasi Stack 1
1. Source Code 1
SOAL 2 5
Batasan 5
Format Input 5
Contoh Kasus 6
A. Source Code 6
B. Output Program 7
C. Pembahasan 8
Soal 3: Implementasi Queue 10
A. Source Code 10
B. Pembahasan 12
SOAL 4 14
Batasan 14
Format Input 14
Contoh Kasus 14
A. Source Code 15
B. Output Program 16
C. Pembahasan 16
TAUTAN GIT 18

DAFTAR GAMBAR

Gambar 1.Screenshot soal 1A 7
Gambar 2.Screenshot soal 1C..... 7
Gambar 3. Screenshot soal 1B..... 7
Gambar 4. Screenshot soal 2A 16
Gambar 5. Screenshot soal 2C..... 16
Gambar 6. Screenshot soal 2B..... 16

DAFTAR TABEL

Tabel 1. Source Code Stack.h 1
Tabel 2. Source Code Stack.cpp 1
Tabel 3. Source Code Prob1.cpp 6
Tabel 4. Source Code Queue.h 10
Tabel 5. Source Code queue.cpp 11
Tabel 6. Source code Probb2.cpp 15

Soal 1: Implementasi Stack

Pada file `stack.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `stack.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

1. Source Code

Tabel 1. Source Code Stack.h

```
#ifndef STACK_H
#define STACK_H

#include <stdexcept>

#define MAX 100

struct Stack {
    int data[MAX];
    int top = -1;
};

void init(Stack* s);
bool isEmpty(const Stack* s);
bool isFull(const Stack* s);
void push(Stack* s, int value);
void pop(Stack* s);
int peek(const Stack* s);

#endif
```

Tabel 2. Source Code Stack.cpp

```
#include "stack.h"

void init(Stack* s) { s->top = -1; }
bool isEmpty(const Stack* s) { return s->top == -1; }
bool isFull(const Stack* s) { return s->top == MAX - 1;
}
```

```
6.
7.  void push(Stack* s, long long value) {
8.      if (isFull(s)) throw std::overflow_error("Stack
    penuh!");
9.      s->data[++s->top] = value;
10. }
11.
12. void pop(Stack* s) {
13.     if (isEmpty(s)) throw std::underflow_error("Stack
    kosong!");
14.     s->top--;
15. }
16.
17. long long peek(const Stack* s) {
18.     if (isEmpty(s)) throw std::underflow_error("Stack
    kosong!");
19.     return s->data[s->top];
20. }
```

B. Pembahasan

1. Pembahasan Stack.h

Pada baris [1] dan [2] itu terdapat *#ifndef STACK_H* dan *#define STACK_H* yang mana keduanya merupakan header guard. Header guard digunakan untuk mencegah file stack.h dibaca lebih dari satu kali oleh compiler. Cara kerjanya adalah ketika file ini pertama kali di-include, STACK_H akan didefinisikan. Kalau file ini di-include lagi di tempat lain, compiler bakal ngecek apakah STACK_H sudah didefinisikan, dan kalau sudah maka isi file ini akan dilewati begitu saja. Di bagian paling akhir file terdapat #endif yang menjadi penutup dari header guard ini.

Selanjutnya Pada baris [3] terdapat *#include <stdexcept>* yang mana digunakan untuk bisa memakai exception handling seperti std::overflow_error dan std::underflow_error di dalam stack.h maupun stack.cpp.

Pada baris [4] terdapat *#define MAX 100005* yang mana digunakan untuk mendefinisikan konstanta MAX dengan nilai 100005. Nilai ini dipakai sebagai ukuran maksimal array yang menyimpan data stack, artinya stack ini bisa menampung hingga 100005 elemen sekaligus. Nilai ini lebih besar dari sebelumnya karena disesuaikan dengan kebutuhan soal yang datanya lebih banyak.

Pada baris [5] sampai [8] terdapat *struct Stack* yang mana digunakan untuk mendefinisikan struktur data Stack. Di dalamnya terdapat dua komponen yaitu *data[MAX]* yang merupakan

array bertipe long long berukuran 100005 untuk menyimpan semua nilai di dalam stack, dan top yang merupakan variabel bertipe int yang menyimpan indeks elemen paling atas stack. Berbeda dengan sebelumnya, kali ini top bukan pointer melainkan integer biasa yang menyimpan indeks.

Pada baris [9] sampai [14] terdapat deklarasi fungsi-fungsi yang akan diimplementasikan di stack.cpp, yaitu *init, isEmpty, isFull, push, pop, dan peek*. Perlu diperhatikan bahwa kali ini tipe data yang digunakan adalah long long bukan int, hal ini karena nilai yang disimpan bisa lebih besar sehingga butuh tipe data yang lebih lebar.

2. Pembahasan tentang Stack.cpp

Pada baris [1] terdapat *#include "stack.h"* yang mana digunakan untuk menyertakan file stack.h ke dalam stack.cpp agar bisa mengakses definisi struct Stack, konstanta MAX, dan deklarasi fungsi-fungsinya.

Pada baris [2] terdapat fungsi *init()* yang mana digunakan untuk menginisialisasi stack sebelum dipakai. Caranya adalah dengan mengatur nilai top menjadi -1 yang menandakan bahwa stack dalam keadaan kosong. Berbeda dengan implementasi sebelumnya yang memakai pointer, kali ini top cukup diset ke -1 karena bertipe integer. Kompleksitas waktu dan ruang fungsi ini adalah O(1).

Pada baris [3] terdapat fungsi *isEmpty()* yang mana digunakan untuk mengecek apakah stack sedang kosong atau tidak. Caranya adalah dengan mengecek apakah nilai top sama dengan -1. Kalau iya, berarti belum ada elemen yang tersimpan dan fungsi bakal mengembalikan nilai true. Kompleksitas waktu dan ruang fungsi ini adalah O(1).

Pada baris [4] terdapat fungsi *isFull()* yang mana digunakan untuk mengecek apakah stack sudah penuh atau belum. Caranya adalah dengan mengecek apakah nilai top sudah sama dengan MAX - 1. Kalau iya, berarti tidak ada lagi slot yang tersedia dan fungsi bakal mengembalikan nilai true. Kompleksitas waktu dan ruang fungsi ini adalah O(1).

Pada baris [5] sampai [8] terdapat fungsi *push()* yang mana digunakan untuk memasukkan nilai baru ke dalam stack. Pertama-tama fungsi ngecek dulu apakah stack sudah penuh pakai isFull. Kalau penuh, fungsi bakal melempar exception std::overflow_error sebagai tanda bahwa operasi tidak bisa dilakukan. Kalau belum penuh, nilai top dinaikkan satu langkah terlebih dahulu menggunakan ++s->top, lalu nilai baru langsung disimpan pada indeks tersebut dalam satu baris sekaligus. Kompleksitas waktu dan ruang fungsi ini adalah O(1).

Pada baris [9] sampai [12] terdapat fungsi *pop*() yang mana digunakan untuk menghapus elemen paling atas dari stack. Pertama-tama fungsi ngecek dulu apakah stack kosong pakai isEmpty. Kalau kosong, fungsi bakal melempar exception std::underflow_error sebagai tanda bahwa operasi tidak bisa dilakukan. Kalau tidak kosong, nilai top cukup dikurangi satu tanpa perlu menghapus data secara fisik, karena elemen tersebut nantinya bakal tertimpa sendiri kalau ada operasi push berikutnya. Kompleksitas waktu dan ruang fungsi ini adalah O(1).

Pada baris [13] sampai [16] terdapat fungsi *peek()* yang mana digunakan untuk mengambil nilai elemen paling atas stack tanpa menghapusnya. Pertama-tama fungsi ngecek dulu apakah stack kosong pakai isEmpty. Kalau kosong, fungsi bakal melempar exception std::underflow_error sebagai tanda bahwa operasi tidak bisa dilakukan. Kalau tidak kosong, fungsi langsung mengembalikan *nilai s->data[s->top]* yaitu elemen yang berada di indeks top saat ini tanpa mengubah posisi top sama sekali. Kompleksitas waktu dan ruang fungsi ini adalah O(1).

SOAL 2

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:
- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:
- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \le n \le 100$
- $-1000 \le A_i \le 1000$

Keterangan:
- n adalah jumlah simbol dalam ekspresi
- A_i adalah nilai bilangan pada input
- Simbol yang valid hanya:
  - bilangan bulat
  - operator: +, -, *, /

Format Input

```
n A1 A2
... An
```

Keterangan:
- Baris pertama `n` menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol `A` sebanyak `n` buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

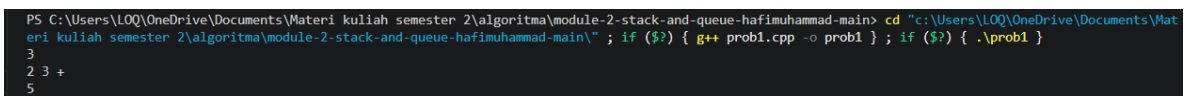
A. Source Code

Tabel 3. Source Code Prob1.cpp

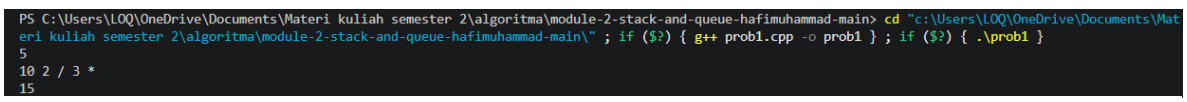
```
#include <iostream>
#include <string>
#include "stack.h"

using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;

    Stack st;
    init(&st);

    for (int i = 0; i < n; i++) {
        string token;
        cin >> token;

        if (token == "+" || token == "-" || token == "*" || token == "/") {

            int b = peek(&st);
            pop(&st);
            int a = peek(&st);
            pop(&st);

            int result;
            if (token == "+") result = a + b;
```

```
            else if (token == "-") result = a - b;
            else if (token == "*") result = a * b;
            else result = a / b;

            push(&st, result);
        } else {
            push(&st, stoi(token));
        }
    }

    cout << peek(&st) << endl;

    return 0;
}
```

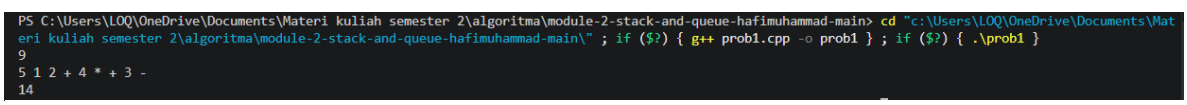
B. Output Program



Gambar 1.Screenshot soal 1A



Gambar 3. Screenshot soal 1B



Gambar 2.Screenshot soal 1C

C. Pembahasan

Pada baris [1] sampai [3] terdapat beberapa include yang mana masing-masing punya fungsi berbeda. *#include <iostream>* digunakan untuk bisa memakai cin dan cout, *#include <string>* digunakan untuk bisa memakai tipe data string, dan *#include "stack.h"* digunakan untuk menyertakan file stack.h agar fungsi-fungsi stack seperti init, push, pop, dan peek bisa dipakai.

Pada baris [7] dan [8] ini terdapat *ios_base::sync_with_stdio(false)* dan *cin.tie(NULL)* yang mana keduanya digunakan untuk mempercepat proses input dan output agar program bisa berjalan lebih cepat terutama saat memproses data yang banyak.

Pada baris [11] dan [12] terdapat *cin >> n* yang mana digunakan untuk membaca jumlah simbol yang akan diproses oleh mesin hitung.

Pada baris [14] sampai [15] terdapat Stack st dan *init(&st)* yang mana digunakan untuk membuat variabel stack bernama st lalu menginisialisasinya agar siap dipakai.

Pada baris [17] dan [18] terdapat perulangan sebanyak n kali yang mana digunakan untuk membaca dan memproses setiap simbol satu per satu. Setiap simbol dibaca sebagai string dan disimpan ke variabel token.

Pada baris [20] terdapat pengecekan apakah token yang dibaca merupakan operator yaitu **+, -, *,** atau **/.** Kalau iya maka blok di dalamnya yang akan dijalankan, kalau tidak berarti token tersebut adalah bilangan.

Pada baris [22] sampai [25] terdapat proses pengambilan dua nilai dari stack. Pertama nilai paling atas diambil pakai peek lalu disimpan ke variabel b, kemudian dihapus pakai pop. Setelah itu nilai berikutnya diambil lagi pakai peek dan disimpan ke variabel a, lalu dihapus pakai pop. Urutan pengambilan ini penting karena b adalah operan kanan dan a adalah operan kiri.

Pada baris [27] sampai [32] terdapat proses perhitungan yang mana dilakukan sesuai dengan operator yang dibaca. Kalau + maka a ditambah b, kalau - maka a dikurangi b, kalau * maka a dikali b, dan kalau / maka a dibagi b. Hasil perhitungan disimpan ke variabel result.

Pada baris [34] terdapat *push(&st, result)* yang mana digunakan untuk memasukkan hasil perhitungan kembali ke dalam stack agar bisa dipakai oleh operasi berikutnya.

Pada baris [35] sampai [37] terdapat blok else yang mana dijalankan kalau token bukan operator melainkan bilangan. Token yang berupa string diubah dulu menjadi integer pakai stoi, lalu langsung dimasukkan ke dalam stack pakai push.

Pada baris [40] terdapat *cout << peek(&st)* yang mana digunakan untuk mencetak hasil akhir. Setelah semua simbol selesai diproses, nilai yang tersisa di dalam stack merupakan hasil akhir dari seluruh perhitungan.

Soal 3: Implementasi Queue

Pada file `queue.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `queue.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 4. Source Code Queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#include <stdexcept>

#define MAX 100

struct Queue {
    int data[MAX];
    int front_idx;
    int rear_idx;
    int count;
};

void init(Queue* q);
bool isEmpty(const Queue* q);
bool isFull(const Queue* q);
void enqueue(Queue* q, int value);
void dequeue(Queue* q);
int front(const Queue* q);
int back(const Queue* q);

#endif
```

Tabel 5. Source Code queue.cpp

```
#include "queue.h"

void init(Queue* q) {
    q->front_idx = 0;
    q->rear_idx = MAX - 1;
    q->count = 0;
}

bool isEmpty(const Queue* q) {
    return q->count == 0;
}

bool isFull(const Queue* q) {
    return q->count == MAX;
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) throw std::overflow_error("Queue penuh!");
    q->rear_idx = (q->rear_idx + 1) % MAX;
    q->data[q->rear_idx] = value;
    q->count++;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) throw std::underflow_error("Queue kosong!");
    q->front_idx = (q->front_idx + 1) % MAX;
    q->count--;
}

int front(const Queue* q) {
    if (isEmpty(q)) throw std::underflow_error("Queue kosong!");
    return q->data[q->front_idx];
}

int back(const Queue* q) {
    if (isEmpty(q)) throw std::underflow_error("Queue kosong!");
    return q->data[q->rear_idx];
}
```

B. Pembahasan

1. Pembahasan Queue.h

Pada baris [1] dan [2] terdapat *#ifndef QUEUE_H* dan *#define QUEUE_H* yang mana keduanya merupakan header guard. Header guard digunakan untuk mencegah file queue.h dibaca lebih dari satu kali oleh compiler. Cara kerjanya adalah ketika file ini pertama kali di-include, `QUEUE_H` akan didefinisikan. Kalau file ini di-include lagi di tempat lain, compiler bakal ngecek apakah `QUEUE_H` sudah didefinisikan, dan kalau sudah maka isi file ini akan dilewati begitu saja. Di bagian paling akhir file terdapat `#endif` yang menjadi penutup dari header guard ini.

Pada baris [4] terdapat *#include <stdexcept>* yang mana digunakan untuk bisa memakai exception handling seperti *std::overflow_error* dan *std::underflow_error* di dalam queue.h maupun queue.cpp.

Pada baris [6] terdapat *#define MAX 100005* yang mana digunakan untuk mendefinisikan konstanta MAX dengan nilai 100005. Nilai ini dipakai sebagai ukuran maksimal array yang menyimpan data queue, artinya queue ini bisa menampung hingga 100005 elemen sekaligus.

Pada baris [8] sampai [13] terdapat `struct` *Queue* yang mana digunakan untuk mendefinisikan struktur data Queue. Di dalamnya terdapat empat komponen yaitu `data[MAX]` yang merupakan array bertipe int untuk menyimpan semua nilai di dalam queue, *front_idx* yang merupakan variabel integer yang menyimpan indeks elemen paling depan, *rear_idx* yang merupakan variabel integer yang menyimpan indeks elemen paling belakang, dan `count` yang merupakan variabel integer untuk menyimpan jumlah elemen yang sedang ada di dalam queue. Berbeda dengan implementasi sebelumnya yang memakai pointer, kali ini queue menggunakan indeks integer dan variabel count sehingga lebih mudah dikelola. Selain itu queue ini menggunakan konsep **circular array** yaitu array yang ujungnya terhubung kembali ke awal sehingga slot yang sudah tidak dipakai bisa digunakan kembali.

Pada baris [15] sampai [21] terdapat deklarasi fungsi-fungsi yang akan diimplementasikan di queue.cpp, yaitu *init, isEmpty, isFull, enqueue, dequeue, front*, dan *back.*

2. Pembahasan Queue.cpp

Pada baris [1] terdapat *#include "queue.h"* yang mana digunakan untuk menyertakan file queue.h ke dalam queue.cpp agar bisa mengakses definisi struct Queue, konstanta MAX, dan deklarasi fungsi-fungsinya.

Pada baris [3] sampai [7] terdapat fungsi *init()* yang mana digunakan untuk menginisialisasi queue sebelum dipakai. Caranya adalah dengan mengatur *front_idx* ke 0 sebagai posisi awal depan, rear_idx ke MAX - 1 sebagai posisi awal belakang, dan *count* ke 0 yang menandakan queue masih kosong. Alasan *rear_idx* diset ke MAX - 1 adalah karena saat enqueue pertama kali, rear_idx akan langsung dihitung ulang menjadi *(MAX - 1 + 1) % MAX = 0*, sehingga elemen pertama tepat masuk di indeks 0. Kompleksitas waktu dan ruang fungsi ini adalah O(1).

Pada baris [9] sampai [11] terdapat fungsi *isEmpty()* yang mana digunakan untuk mengecek apakah queue sedang kosong atau tidak. Caranya adalah dengan mengecek apakah nilai count sama dengan 0. Kalau iya, berarti tidak ada elemen yang tersimpan dan fungsi bakal mengembalikan nilai true. Penggunaan count membuat pengecekan ini lebih simpel dibanding membandingkan posisi front dan rear. Kompleksitas waktu dan ruang fungsi ini adalah O(1).

Pada baris [13] sampai [15] terdapat fungsi *isFull()* yang mana digunakan untuk mengecek apakah queue sudah penuh atau belum. Caranya adalah dengan mengecek apakah nilai count sudah sama dengan MAX. Kalau iya, berarti tidak ada lagi slot yang tersedia dan fungsi bakal mengembalikan nilai true. Kompleksitas waktu dan ruang fungsi ini adalah O(1).

Pada baris [17] sampai [22] terdapat fungsi *enqueue()* yang mana digunakan untuk memasukkan nilai baru ke bagian belakang queue. Pertama-tama fungsi ngecek dulu apakah queue sudah penuh pakai isFull. Kalau penuh, fungsi bakal melempar exception std::overflow_error sebagai tanda bahwa operasi tidak bisa dilakukan. Kalau belum penuh, rear_idx dihitung ulang menggunakan rumus (rear_idx + 1) % MAX yang merupakan ciri khas circular array agar indeks kembali ke awal kalau sudah mencapai batas akhir array. Setelah itu nilai baru disimpan di posisi rear_idx yang baru, lalu count dinaikkan satu. Kompleksitas waktu dan ruang fungsi ini adalah O(1).

Pada baris [24] sampai [28] terdapat fungsi *dequeue()* yang mana digunakan untuk menghapus elemen paling depan dari queue. Pertama-tama fungsi ngecek dulu apakah queue kosong pakai isEmpty. Kalau kosong, fungsi bakal melempar exception std::underflow_error sebagai tanda bahwa operasi tidak bisa dilakukan. Kalau tidak kosong, front_idx dihitung ulang menggunakan rumus (front_idx + 1) % MAX agar mengikuti konsep circular array, lalu count dikurangi satu. Data lama tidak perlu dihapus secara fisik karena nantinya bakal tertimpa sendiri oleh operasi enqueue berikutnya. Kompleksitas waktu dan ruang fungsi ini adalah O(1).

Pada baris [30] sampai [34] terdapat fungsi *front()* yang mana digunakan untuk mengambil nilai elemen paling depan queue tanpa menghapusnya. Pertama-tama fungsi ngecek dulu apakah queue kosong pakai isEmpty. Kalau kosong, fungsi bakal melempar exception std::underflow_error sebagai tanda bahwa operasi tidak bisa dilakukan. Kalau tidak kosong, fungsi langsung mengembalikan nilai q->data[q->front_idx] yaitu elemen yang berada di indeks front_idx saat ini tanpa mengubah posisinya sama sekali. Kompleksitas waktu dan ruang fungsi ini adalah O(1).

Pada baris [36] sampai [40] terdapat fungsi *back()* yang mana digunakan untuk mengambil nilai elemen paling belakang queue tanpa menghapusnya. Pertama-tama fungsi ngecek dulu apakah queue kosong pakai isEmpty. Kalau kosong, fungsi bakal melempar exception std::underflow_error sebagai tanda bahwa operasi tidak bisa dilakukan. Kalau tidak kosong, fungsi langsung mengembalikan nilai q->data[q->rear_idx] yaitu elemen yang berada di indeks rear_idx saat ini tanpa mengubah posisinya sama sekali. Kompleksitas waktu dan ruang fungsi ini adalah O(1).

SOAL 4

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- A_i: elemen array

Format Input

```
n k A1 A2
... An
```

Keterangan:

- Baris pertama `n` menyatakan jumlah atau panjang array. Lalu `k` besar atau jumlah elemen dalam satu kelompok ayng dibentuk
- Baris kedua berisi simbol `A` sebanyak `n` buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
`5 3` `1 2 3 4 5`	`6 9 12`

5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Tabel 6. Source code Probb2.cpp

```
#include <iostream>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n, k;
    cin >> n >> k;

    int arr[100];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    if (k <= 0 || k > n) {
        cout << endl;
        return 0;
    }

    int jumlah = 0;
    for (int i = 0; i < k; i++) {
        jumlah += arr[i];
    }

    cout << jumlah;

    for (int i = k; i < n; i++) {
        jumlah -= arr[i - k];
        jumlah += arr[i];
        cout << " " << jumlah;
    }

    cout << endl;
```

```
34.     return 0;
35. }
```

B. Output Program

```
PS C:\Users\LOQ\OneDrive\Documents\Materi kuliah semester 2\algoritma\module-2-stack-and-queue-hafimuhammad-main> cd "c:\Users\LOQ\OneDrive\Documents\Materi
kuliah semester 2\algoritma\module-2-stack-and-queue-hafimuhammad-main\" ; if ($?) { g++ prob2.cpp -o prob2 } ; if ($?) { .\prob2 }
5 3
1 2 3 4 5
6 9 12
```

Gambar 4. Screenshot soal 2A

```
PS C:\Users\LOQ\OneDrive\Documents\Materi kuliah semester 2\algoritma\module-2-stack-and-queue-hafimuhammad-main> cd "c:\Users\LOQ\OneDrive\Documents\Materi
kuliah semester 2\algoritma\module-2-stack-and-queue-hafimuhammad-main\" ; if ($?) { g++ prob2.cpp -o prob2 } ; if ($?) { .\prob2 }
5 5
1 2 3 4 5
15
```

Gambar 6. Screenshot soal 2B

```
PS C:\Users\LOQ\OneDrive\Documents\Materi kuliah semester 2\algoritma\module-2-stack-and-queue-hafimuhammad-main> cd "c:\Users\LOQ\OneDrive\Documents\Materi
kuliah semester 2\algoritma\module-2-stack-and-queue-hafimuhammad-main\" ; if ($?) { g++ prob2.cpp -o prob2 } ; if ($?) { .\prob2 }
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
```

Gambar 5. Screenshot soal 2C

C. Pembahasan

Pada baris [1] terdapat *#include <iostream>* yang mana digunakan untuk bisa memakai cin dan cout. Perlu diperhatikan bahwa program ini tidak menggunakan queue.h sama sekali, melainkan menyelesaikan masalah sliding window langsung menggunakan array biasa.

Pada baris [6] sampai [7] terdapat *ios_base::sync_with_stdio(false)* dan *cin.tie(NULL)* yang mana keduanya digunakan untuk mempercepat proses input dan output agar program bisa berjalan lebih cepat terutama saat memproses data yang banyak.

Pada baris [9] sampai [10] terdapat *cin >> n >> k* yang mana digunakan untuk membaca dua nilai sekaligus yaitu n sebagai jumlah elemen array dan k sebagai ukuran jendela atau banyaknya elemen yang dijumlahkan dalam satu kelompok.

Pada baris [12] sampai [14] terdapat deklarasi array `arr` bertipe static int berukuran 100005 yang mana digunakan untuk menyimpan seluruh elemen yang dibaca. Kata kunci static digunakan agar array berukuran besar ini dialokasikan di memori statis, bukan di stack, sehingga tidak menyebabkan stack overflow.

Pada baris [15] sampai [18] terdapat pengecekan kondisi tidak valid yaitu ketika nilai k kurang dari atau sama dengan 0, atau ketika k lebih besar dari n. Kalau kondisi ini terpenuhi, program langsung mencetak baris kosong dan berhenti karena tidak mungkin membentuk kelompok yang valid.

Pada baris [20] sampai [23] terdapat perulangan yang mana digunakan untuk menghitung jumlah elemen pada window pertama yaitu dari indeks 0 sampai k-1. Hasilnya disimpan ke variabel `jumlah`.

Pada baris [25] terdapat *cout << jumlah* yang mana digunakan untuk mencetak hasil penjumlahan window pertama tanpa spasi di depannya.

Pada baris [27] sampai [31] terdapat perulangan yang mana digunakan untuk menggeser window satu langkah ke kanan sebanyak n-k kali. Cara kerjanya adalah dengan mengurangi `jumlah` dengan elemen yang keluar dari window yaitu *arr[i - k],* lalu menambahkan elemen baru yang masuk ke window yaitu *arr[i].* Teknik ini disebut sliding window dan jauh lebih efisien dibanding menjumlahkan ulang seluruh elemen setiap kali window bergeser. Hasil penjumlahan window baru langsung dicetak dengan spasi di depannya.

Pada baris [33] terdapat *cout << endl* yang mana digunakan untuk mencetak baris baru setelah semua hasil window selesai dicetak.

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-hafimuhammad.git